

Advancing user-space networking for DDS message-oriented middleware: Further extensions

Vincent Bode ^a *, Carsten Trinitis ^a, Martin Schulz ^a, David Buettner ^b, Tobias Preclik ^b

^a Technical University of Munich, Boltzmannstrasse 3, Munich, 85748, Germany

^b Siemens AG, Munich and Erlangen, Germany

ARTICLE INFO

Keywords:

Data Distribution Service

Real-time middleware

Userspace networking

ABSTRACT

Due to the flexibility it offers, publish–subscribe messaging middleware is a popular choice in Industrial IoT (IIoT) applications. The Data Distribution Service (DDS) is a widely used industry standard for these systems with a focus on versatility and extensibility, implemented by multiple vendors and present in myriad deployments across industries like aerospace, healthcare and industrial automation. However, many IoT scenarios require real-time capabilities for deployments with rigid timing, reliability and resource constraints, while publish–subscribe mechanisms currently rely on components that are not strictly real-time capable, such as the Linux networking stack, making it hard to provide robust performance guarantees without large safety margins.

In order to make publish–subscribe approaches viable and efficient also in such real-time scenarios, we introduce user-space DDS networking transport extensions, allowing us to fast-track the communication hot path by bypassing the Linux kernel. For this purpose, we extend the best-performing vendor implementation from a previous study, CycloneDDS, to include modules for two widespread user-space networking technologies, the Data Plane Development Kit (DPDK) and the eXpress Data Path (XDP). Building on this, we additionally offer two more extensions to the second most performant implementation FastDDS, also based on DPDK and XDP, and realize novel optimizations not present in the original extension implementations. We evaluate each extension's performance benefits against four existing DDS implementations (OpenDDS, RTI Connext, FastDDS and CycloneDDS). The DPDK-based and XDP-based extensions offer a performance benefit of 31%–38% and 18%–22% reduced mean latency, respectively, as well as an increase in bandwidth and sample rate throughput of at least 160%, while reducing the latency bound by at least 93%, demonstrating the performance and dependability advantages of circumventing the kernel for real-time communications.

1. Introduction

Developing flexible, fast and future-proof communication layers for systems is a challenge that many software projects face, with a multitude of project requirements and constraints often complicating the implementation of a seemingly simple software component. This is the reason why industrial systems have been leveraging communications middleware to outsource this burden to off-the-shelf implementations ever since the inception of the concept in 1968 [1]. For the Industrial Internet of Things (IIoT),

* Corresponding author.

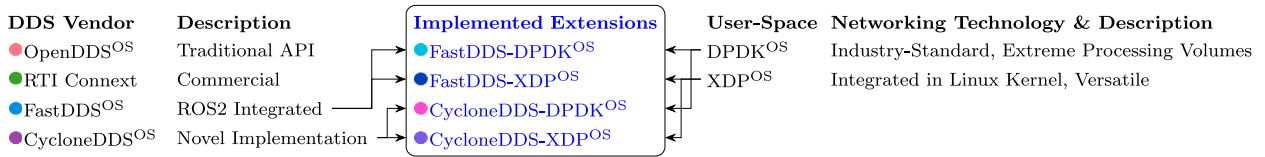
E-mail address: vincent.bode@tum.de (V. Bode).

<https://doi.org/10.1016/j.pmcj.2025.102013>

Received 4 August 2024; Received in revised form 11 December 2024; Accepted 14 January 2025

Available online 27 January 2025

1574-1192/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).



OS: Open-Source technology, facilitating uncomplicated extension compared to closed-source commercial implementations.

Fig. 1. Overview of DDS and User-Space Networking technologies considered when implementing our DDS User-Space Networking extensions. Preference was given to building on technologies that are publicly available as open-source, that leverage modern APIs, and that have an active user base that can benefit from the capabilities of accelerated DDS.

the publish–subscribe architecture is a popular choice for message-oriented middleware because of its ease-of-use and extensibility. There are a multitude of publish–subscribe technologies, but one standard gaining traction is the Data Distribution Service (DDS), an API and protocol specification for distributed data-centric message exchange maintained by the Object Management Group [2] and implemented in open-source and commercial systems backed by a large variety of vendors.

DDS as a messaging middleware features multiple benefits over the traditional socket communication model, including reduced coupling and increased dependability/extensibility by means of abstraction and encapsulation, as well as transparently handling communication life-cycle management, message tracking and retransmission. All of these make DDS an attractive candidate for adoption in IIoT scenarios, with deployments including robotics systems (via the integration into the Robot Operating System ROS2) [3], military communications [4], mobile medical diagnosis equipment [5], and factory automation [6,7].

However, in many use cases we face reliability and timing restrictions, which requires every component in the system to offer real-time and reliability guarantees for the system as a whole to be considered dependable. This runs contrary to the pressure of adding ever more features into control systems, leading them to be more versatile at the cost of complexity. The latest push of integrating machine learning into time-constrained systems only adds to the difficulty of guaranteeing boundedness and reliability. These timing constraints are also a problem for most communication frameworks, including DDS implementations, as off-the-shelf middleware generally relies on technologies that do not offer real-time functionality or timing guarantees natively, like the Linux networking stack [8]. Despite this, DDS has found its way into many timing-critical applications, where careful configuration of DDS is necessary in addition to using ample performance safety margins, as only this ensures safe system operations in both normal and adverse conditions. However, further adoption is hindered by a lack of studies showing reliable system operation under unfavorable conditions, making it harder for application developers to trust in the middleware’s resilience. We aim to advance this area of soft-real-time guarantees by reducing the number of real-time incapable components on the transmission path, allowing us to shrink the safety margins and with that improve overall performance, reliability and thus trustworthiness.

In this paper, we evaluate the existing soft real-time capabilities of four vendors and extend two DDS vendor implementations to include support for two tried-and-tested user-space networking technologies each, drastically improving latency bounds and reducing performance variability while further enhancing other key performance metrics. We offer the following contributions:

- Analysis of performance considerations and software engineering requirements when combining DDS with user-space networking technologies,
- Extension of the two best-performing vendor implementations CycloneDDS and FastDDS to include support for two major user-space networking technologies DPDK and XDP, improving its soft real-time characteristics,
- Evaluation of the reliability and performance benefits of the four DDS user-space networking extensions against four regular DDS implementations using DDS-Perf cross-vendor benchmarks.

To survey the state-of-the-art performance, we selected four DDS implementations that are widely adopted in the DDS community and that emphasize applications in IIoT scenarios (Fig. 1). Apart from their significant total market share, vendors developing these systems specifically advertise system performance as a selling point. The first system, OpenDDS, is maintained by Object Computing, and is open-source. OpenDDS is one of the original implementations that was available when the original DDS standard was released in 2004, and thus offers only the traditional C++ DDS API. RTI Connex by Real Time Innovations is a performance-oriented but closed-sourced commercial implementation, which offers the modernized C++11 DDS API alongside the traditional. Further open-source implementations examined include FastDDS (eProsima), which is integrated as the default DDS vendor in the Robot Operating System 2 (ROS2) and thus facilitates a lot of industrial IoT deployments, and the relatively new CycloneDDS (Eclipse Foundation), which is still maturing but shows promise and is already attracting a growing user base.

We chose two user-space networking technologies, DPDK and XDP, to demonstrate the usefulness of user-space networking for DDS due to their suitability for (Industrial-) IoT applications: (1) Data-Plane Development Kit (DPDK), a user-space networking technology maintained by the Linux Foundation and often referred to as the de facto standard for acceleration of packet processing [9], which relies on specialized drivers to bypass the kernel and deliver packets directly to DPDK-enabled applications; and (2) eXpress Data Path (XDP), a Linux kernel mechanism that uses Berkeley Packet Filters (BPFs) to do in-kernel packet processing outside the regular network stack [10]. The former is ubiquitous in performance packet processing and the latter was recently mainstreamed into the Linux kernel, thus both options are production-ready and widely available. We will be examining performance

aspects of using the chosen network technologies with DDS throughout this paper. To the best of our knowledge, we offer the first implementations of user-space networking extensions for any DDS system, which is available to the public as open-source.¹

The remainder of this paper is structured as follows: We first introduce DDS and examine its current real-time capabilities in Section 2. The case for user-space networking and the two chosen technologies is made in Section 3. In Section 4, we cover implementation aspects of the CycloneDDS and FastDDS extensions and subsequently evaluate the performance obtained in Section 5. Finally, in Section 6 we discuss the results and possibilities for future work.

2. DDS and current real-time capabilities

The advantages of the publish–subscribe paradigm have helped it spread throughout communications technologies, with both middleware systems like MQTT and protocols like IGMP adopting the pattern. It supports a strong decoupling of sender and receiver, which makes it appealing for dynamic networks of distributed systems. DDS is a pub-sub middleware standard built for a very wide range of use-cases, and it brings the necessary configurability and adaptability to suit many applications. Due to implementations and backing by over a dozen vendors and two decades of standard updates, the range of deployments has grown and now pervades many sectors. Unlike many of its competitors, DDS also does not rely on a centralized broker or message queue but functions fully distributed. Because industrial use-cases often prioritize reliability, performance and resource-consumption requirements, the ability to pick from a diverse set of implementations makes DDS attractive for projects that need solutions tailored to their needs. This diversity comes at the cost of complexity, though, as assessing the capabilities of DDS as a whole requires studying a range of implementations. To truly evaluate the performance benefits of user-space networking for DDS, we will consider four major DDS implementations, covering both commercial and open source implementations.

2.1. Previous work

From the beginnings of DDS with the standard’s finalization in 2004 [11], OMG has continued to norm the DDS ecosystem, publishing three further core standards and 15 auxiliary standards. These all regulate facets of the DDS ecosystem today, specifying anything from programming language specific APIs to wire formats for interoperability. It is not uncommon, however, to find vendor-specific extensions in place of standardized mechanisms, as these often predate standardization. Because of the lack of an official compliance certification process for DDS, benchmarking DDS implementations is fairly challenging. Thus, cross-vendor benchmarking tools have been developed, like DDS-Perf [12] to hide the complexity of dealing with multiple DDS vendors and to enable the evaluation of performance in a platform-independent way.

In the past, other publish–subscribe messaging standards, like OPC UA or ZeroMQ, have been assessed against one or more DDS vendors [13,14]. However, these studies lack any analysis of specialized tuning for real-time aspects, such as latency bounds. Previous work has also examined higher-level protocols built on top of DDS, such as ROS2 for robot communication [3]. An investigation was conducted to evaluate DDS performance in the real-time context [15], which found a reduction in latency jitter was possible by changing network driver settings, however, it focuses on just a single and unnamed DDS vendor and a specific network card. Still others have benchmarked the performance of different optional standards of the DDS ecosystems, like DDS-XRCE, for resource constrained environments [16], or the virtualization overhead of containerizing DDS [17]. Many comparison studies are vendor driven, but in 2013 an independent survey compared two DDS libraries to each other (OpenSplice and RTI) [18].

Surprisingly, to the best of our knowledge there have been no efforts to examine combining DDS with user-space networking technologies, despite their well-known advantages of minimizing kernel interference in communication for latency and bandwidth [10], and the widespread adoption in other disciplines, such as high-performance computing. Some adjacent efforts are currently ongoing at the time of writing to harmonize DDS with Time-Sensitive Networks (TSNs) [19], however, TSNs are a much more involved solution that requires extensive hardware and software support to provide deterministic Ethernet behavior. In contrast, user-space networking is implemented mostly in software, leading to deployability advantages. Additionally, while high hopes are placed on DDS-TSN and some vendors already claim TSN support [20], these developments are in very early stages and currently lack production readiness. In fact, we have not yet been able to reproduce any DDS vendor’s TSN demonstration. Meanwhile, DPDK and XDP are very well established user-space networking technologies, and can be seen as a compromise between highly specialized TSN and widely-deployed traditional Linux networking. It is here that we can start to explore accelerating DDS using DPDK and XDP.

In our original study on user-space networking for DDS [21], of which this work is an extension, we implemented and evaluated two DDS user-space networking extensions for CycloneDDS. We now improve on this by implementing two further user-space networking extensions for a different DDS vendor – FastDDS – and reevaluating our solution, which allows us to compare the individual technologies involved, both on the differences between DPDK and XDP, as well as the differences between CycloneDDS and FastDDS. Additionally, we further implement several new optimizations into the FastDDS extensions, demonstrating further performance gains, some of which we were able to backport to the CycloneDDS extensions. We make all extensions available as open-source and offer a discussion on how the technologies could be further integrated for performance and ease-of-use.

¹ Git Repositories: <https://github.com/caps-tum/cyclonedds-dpdk-xdp> and <https://github.com/caps-tum/fastdds-dpdk-xdp>.

2.2. The problem: Latency bounds

Previously, Bode et al. conducted performance studies on DDS using DDS-Perf across the four vendors. There is a general performance study available [12], as well as a real-time focused study [22]. Throughout the real-time focused study, several hardware- and software-based tuning techniques were applied to optimize the out-of-the-box performance of each vendor, targeting the reference performance metrics provided by DPDK's Layer 2 benchmarking tool `l2reflect`. While they were able to bring down average latencies to around 100 μ s for the better performing systems (CycloneDDS, followed by FastDDS), the maximum observed latencies among 3 million packets were still an order of magnitude higher, with rare but significant latency spikes of up to 4000 μ s for the worst performing system (OpenDDS) even after tuning. While this far outperforms the out-of-the-box latency bounds (OpenDDS: 17,400 μ s), a latency outlier will still observe up to 40 $\times$ the mean observed latency [22].

A 0.0001% (or one-in-a-million) chance of a deadline missed by 4 ms might seem insignificant, but this is rather unacceptable for many types of industrial control systems. Consider a piston engine controller that actuates ignition of the combustion chamber, which is traditionally implemented using hard real-time technologies. The controller needs to actuate the ignition according to the input parameters with very reliable timing, potentially thousands of times per second. At a signal rate of 1 kHz (1000 occurrences per second), an outlier on a soft real-time system – such as one building on DDS – would occur once every 15 min, which can cause either service outage (engine stall) or potential damage to the machinery through, e.g., misfires if the delayed signal arrival is not handled correctly. Consequences are not just limited to property damage: a similar mean time between failures (MTBF) to meet the deadline in a life-support system, such as an oxygen flow monitor, would mean opening a window for undetected dangerous conditions, failing to trigger necessary alarms and thus posing obvious health and safety risks. While redundancy is often employed to reduce risks of failure, it can only try to mitigate the effects, but cannot eliminate the root cause of the failure. Clearly, both the rate of occurrence of outliers and their magnitude need to be further reduced before we can consider introducing DDS as a soft real-time technology suitable for these applications traditionally reserved for hard real-time systems.

2.3. The motivation for soft real-time

One might question why soft real-time implementations are being considered for these applications when the problem of latency bound guarantees is already solved by conventional hard real-time systems. It turns out that hard real-time systems have drawbacks that drive implementation and maintenance costs, which are not present in soft real-time systems:

- A hard real-time system requires specialized hardware and software to be able to offer guaranteed latency bounds. Often, this means having to use a certain embedded board packaged with a specific compatible real-time operating system (RTOS). These are significantly more expensive to write code for than commodity hardware/software due to the necessarily restrictive nature of the technologies involved to provide bounded response times [23]. Additionally, boards with real-time capabilities simply cost more than their commodity counterparts, a factor which compounds when production quantities need to be increased.
- Apart from the effort required to write the code itself, significant resources also need to be invested into runtime estimations so that a feasible schedule can be found to ensure the latency bound. This is a well-known problem, which still requires substantial manual labor, and it is amplified by the necessity to repeat this procedure for any modifications that are made to a system throughout its development cycle [24].
- Hard real-time systems have a tendency to be much more static once they are deployed than their commodity counterparts [25]. Often, they forego hardware and software features not strictly necessary for operation, which would otherwise make verification of real-time properties more difficult, in favor of keeping things simple. While this can be an advantage for dependability, making changes to an existing system is difficult due to the tighter integration and coupling between components. Thus, providing support and rolling out new features is not as simple or cost-effective as providing Over-The-Air (OTA) updates common in the commodity software world, but instead requires more complex procedures, such as re-flashing (E)EPROMs.

We observe that the drawbacks of hard real-time systems all boil down to economical considerations. In the end, the reason why real-time systems are restricted to a small fraction of all software deployments, even though their advantages could benefit other types of software as well, is simply the question of cost-effectiveness. This implies that the closer that soft real-time solutions get to emulating the performance of a hard real-time system, the more interesting of an alternative they become for users to deploy.

3. The case for user-space networking

Because traditional real-time applications are implemented close to the hardware, they often ship with embedded technology stacks that are lightweight and purpose-built. DDS, however, not being a technology aimed solely at real-time applications and their constraints, sits on top of a much more heavy-weight technology stack, with a typical setup relying on a fully fledged operating system (in our case Jupiter, a real-time flavor of Linux) and the applications and drivers that come with it. The ability to use commodity hardware and software offers a multitude of advantages for development and operations/maintenance, but each additional layer of abstraction and isolation makes it more difficult to provide reliability and timing guarantees across the entire stack. Furthermore, technology stacks are growing in size and complexity, with the advent of connected, smart and AI-enabled devices only adding to the existing difficulties. Whereas an application running on a programmable microcontroller typically has

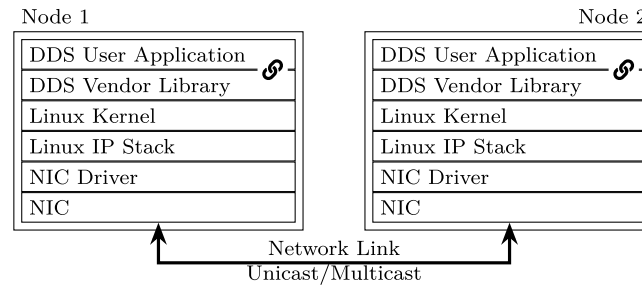


Fig. 2. The technology stack that a DDS sample (DDS term for a message/packet) needs to traverse downwards on the publisher and upwards on the receiver. The many software layers make it difficult to provide timing or reliability guarantees as each layer can introduce unexpected delays.

direct and immediate hardware access to its communication facilities, a message transmission on a typical server system traverses many layers of software and hardware before arriving at its destination.

When looking at the technology stack that a sample (DDS term for a message/packet) needs to traverse during delivery (Fig. 2), it may be surprising that the DDS implementations can even reach the average performance of real-time oriented technologies as demonstrated by Bode et al. [22]. Especially the path through the kernel is often expensive, as the write and read system calls cause processor mode switches and management overhead that make the call alone around 100-500 times more expensive than a regular function call on x86-64 [26], and that overhead is incurred before the kernel even starts doing anything useful. Afterwards, the packet needs to traverse the kernel I/O routines, the network stack and the NIC driver before being sent off over the wire. Repeating this process thousands of times per second is therefore not only expensive, but also prone to interference from resource contention, which can cause critical delays in delivery and unexpected packet delay variation. Many DDS systems take steps to minimize the effects of this problem, which often amount to batch processing to minimize overhead. For example, CycloneDDS uses vectored I/O using the `readv` and `writv` system calls to reduce the total number of system call invocations. FastDDS uses locator mechanisms to send multiple packets to individual destinations in one go, thus also reducing overhead. However, the problem remains that all packets need to eventually traverse the kernel and its real-time incapable network stack twice on the critical transmission path, or four times when a round-trip is necessary to handle, e.g., acknowledgments during reliable message delivery.

DDS is not the first communication system that encounters this problem. Other disciplines, such as high performance computing or high-throughput packet processing, have already encountered similar issues with performance-critical networking. In fact, there are patches to Linux that introduce real-time functionality for aspects of the kernel, improving on e.g. scheduling and interrupt handling, but these alone are not sufficient to guarantee latency bounds for network communication. This is due to the fact that, e.g., the network stack does not provide timing guarantees even with Linux's real-time patches due to the unpredictability of interrupt handling, buffering and queuing effects. Even if Linux were able to provide timing guarantees across all its software components, the end-to-end boundedness of packet processing times would still rely on driver and hardware cooperation as well as the compatibility of the actual user-space applications to ensure that every piece of the critical path can support the real-time requirements. Since enforcing latency boundedness in large monolithic pieces of software like the Linux kernel is difficult, eventually the idea was born to circumvent the kernel entirely and thus cut out several layers of software from the critical path. This technology is now known as user-space networking, allowing a regular user-space application or library (such as DDS) to communicate directly with the hardware in user-level code without involving the kernel and its related security mechanisms (except for the initial access control and setup, which is still secured by the kernel). While this has obvious performance improvement potential, it comes at the cost of losing many of the services the kernel offers in the first place, such as resource sharing, isolation, security or safety measures, and memory management. Despite these drawbacks, user-space networking is still viable for specialized deployments, especially in scenarios where dedicated resources can be made available, such as in high performance computing or embedded use-cases. To enable users of DDS to leverage the advantages offered by user-space networking technologies, we adapted two high-performance DDS implementations, CycloneDDS and FastDDS, to work with the DPDK and XDP user-space networking technologies to examine potential performance and reliability advantages. We implemented this functionality as two extension modules, CycloneDDS-DPDK and CycloneDDS-XDP, that operate directly on Layer 2 Ethernet to facilitate hardware-accelerated message exchange. Subsequently, these two extensions were ported to a different DDS vendor FastDDS, and we integrated some novel optimizations not present in the original publication of the CycloneDDS extensions.

3.1. DPDK

The Data-Plane Development Kit (DPDK) is a user-space networking technology originally developed by Intel and now maintained by the Linux foundation [27]. It relies on specialized NIC drivers that allow it to circumvent the regular Linux networking stack and still take advantage of hardware capabilities like offloading. As such, DPDK applications usually have exclusive access to the network interface (although mechanisms for sharing exist) and the NIC is practically invisible to Linux and any other networked application. A DPDK driver is referred to as a Poll-Mode Driver (PMD), as it largely avoids expensive interrupts in favor of polling for incoming messages in a run-to-completion/batch processing model. DPDK is frequently deployed in high-performance packet processing applications with processing requirements beyond 10 million packets per second [28].

Table 1

Comparison of merits between DPDK & XDP user-space networking technologies.

Factor	DPDK	XDP
Portability	– HW & driver support essential	+ Can function solely in-kernel
Deployability	– Disruptive, complex, requires special drivers, hardware support and root privileges	+ Comparatively simple, but root required, project still maturing but integrated in mainline Linux
Interoperability	~ Bifurcation setup complex, disruptive	+ Built-in, simple, less intrusive
Performance	+++ De-facto standard	+ Improvements over regular kernel networking
Recommended Use-case	Specialized deployments with extreme processing requirements	Performance oriented deployments prioritizing usability

3.2. XDP

The eXpress Data Path (XDP) [10] is an in-kernel packet processing mechanism that can also be used to implement user-space networking. It allows programs to inject a Berkeley Packet Filter (BPF) into the kernel that executes packet processing logic like routing, rewriting or replying. BPFs can also selectively redirect packets into user-space programs before they enter the Linux network stack, which we use to deliver DDS packets directly to our XDP extensions. With sufficient hardware and software support, BPFs can run directly on the NIC using offloading, as BPFs are compiled to specialized byte code using LLVM designed to be Just-In-Time (JIT) translated for the NIC at runtime. As a fallback, BPFs can run either in the driver or in the kernel if no NIC hardware or software support is offered at all. Due to its integration with the Linux kernel, XDP-enabled applications can be deployed on virtually any modern Linux system.

3.3. Comparison

While both technologies shift kernel responsibilities to the application developers in order to improve performance, their underlying design philosophies make them distinct (Table 1). Of these, the following trade-offs are relevant for users developing and deploying applications:

Portability. While DPDK requires special drivers that need to be managed by the user, XDP has more widespread support due to its integration in the Linux kernel. In fact, XDP programs can run either on the NIC directly, in the NIC driver, or in the Linux kernel as a fallback depending on the hardware and software support available. Thus, XDP can live entirely without hardware and driver support if necessary, albeit at the cost of performance. An XDP enabled program can be shipped like any other Linux application, and it can load the BPF program into the kernel by itself given sufficient privileges. Meanwhile, DPDK is a much more disruptive solution: while the application itself can be delivered as usual, a working DPDK setup requires the user or administrator to work through an 11-chapter “Getting Started” guide [27] to set up libraries, drivers, and kernel modules and to manually configure the NICs by hand, which is a much more involved process. We also note that the DPDK API is evolving, which means our DPDK extensions are only compatible with specific versions of DPDK and need to be ported to the version supported by the target system if they are to be redeployed.

Interoperability. The exclusive NIC access required by DPDK is another potential issue for users looking to adopt a user-space networking technology. While special bifurcation drivers exist [27], allowing DPDK applications to share the NIC with regular applications that require Linux network functionality given that hardware support is available, the setup is complex. Even sharing the network interface with a second DPDK-enabled application requires application support and special configuration. Meanwhile, user and kernel space networking coexistence is supported out of the box with XDP’s BPFs, with a BPF only acting as intermediate logic between the hardware and the kernel. We use this functionality to redirect only packets that match each vendor’s XDP-L2 extension’s `ethertype`, letting all other traffic take its regular path through the kernel. Note that in both cases though, DPDK/XDP gain full access to network traffic, and can even reroute or respond to other applications’ traffic.

Performance. While the above aspects generally weigh in favor of XDP, DPDK’s strong point is performance. Performance aspects were considered to be so important for implementing DPDK, that the manual even includes information about how many cache lines are loaded per packet and which lines need to be cached for efficient access to header fields. In fact, developers of XDP are using DPDK as a performance benchmark [29] to optimize their implementations. DPDK is often referred to as the de-facto standard for high performance packet processing, and has been successfully applied to applications, like the implementation of virtual switches [30]. Because of this, we expect DPDK to significantly outperform XDP as a transport layer for DDS, at the cost of the aforementioned drawbacks.

Due to the slightly different use-cases for these technologies, we decided to adopt both technologies for use in DDS. This will allow us to compare their suitability against each other and the regular Linux IP stack, while also allowing users to choose the right technology based on their requirements. Additionally, since we implement both extensions for both CycloneDDS and FastDDS, we can observe whether the specifics of each DDS implementation have an effect on the performance of the user-space networking library.

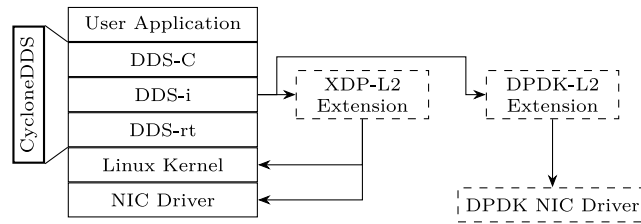


Fig. 3. The CycloneDDS system stack. We intercept messages at the implementation layer (DDS-i) for both the XDP-L2 extension (center) and the DPDK L2 extension (right). In the case of XDP, the extension forwards it through an XSK socket to the kernel or NIC driver (depending on the hardware and software support). On the other hand, the DPDK extension communicates directly with the specialized DPDK driver.

4. Implementation

For our initial study of user-space networking in DDS [21], we chose CycloneDDS as our base implementation. We preferred open-source DDS implementations, as these would be easier to extend and debug as opposed to closed-source commercial offerings. We also prefer support for the modern C++ API and a lack of support of the traditional API, as this allows us to leverage C++11 features and spares us from having to worry about compatibility layers for the traditional API.

Compared to some traditional DDS systems, CycloneDDS is one of the most modern implementations of the DDS standard. Among the open source systems, it offers several advantages when implementing new transport technologies, the most important being that CycloneDDS is structured into layered components (Fig. 3). The uppermost layer (DDS-C) consists of user-facing APIs specified by the DDS standard, as well as CycloneDDS specific extensions. The DDS-C layer then interfaces with the implementation layer (DDS-i), which includes most of the business logic required to implement a DDS system. The final layer is the runtime layer (DDS-rt), which handles platform and environment specifics, such as the implementation differences between Linux and other operating systems as well as different levels of hardware capability.

The layered system allows us to effectively intercept the internal API calls made by CycloneDDS without too many changes to the core CycloneDDS source code itself. Besides its good performance in the previous studies [22], this was a key reason why CycloneDDS is our first choice for implementing user-space networking extensions. We choose to inject the DPDK-L2 and XDP-L2 extension code near the lower end of the DDS-i layer.

As part of this extended study, we added a second vendor to implement user-space networking extensions for. While FastDDS is not quite as novel as CycloneDDS, it is widely adopted due to it being the default communications vendor in the Robot Operating System 2 (ROS 2). Developers working in the real-time field are therefore more likely already familiar with this DDS implementation. It also showed the best or second-best latency performance in previous studies [12,22]. Akin to CycloneDDS, FastDDS is similarly structured into components, with the principal components including FastDDS and FastRTPS. Each of these is further subdivided into subcomponents responsible for aspects such as security, flow control and data sharing. Fortunately, FastDDS was built with extensibility in mind, it includes a transport interface (even with accompanying documentation) that allows users to provide their own implementations of underlying communication transports. Taking inspiration from the existing transport for UDP, we thus implement the interface and make minor changes to related components to provide the necessary functionality for the DPDK and XDP extensions.

Inserting the extensions in the CycloneDDS DDS-i layer and at the FastDDS transport interface allows us to take advantage of the following properties:

4.1. Automatic entity lifecycle management

The CycloneDDS DDS-i layer implements the discovery of remote participants (using the standardized DDS Simple Participant Discovery Protocol, SPDP) and the discovery of remote endpoints (SEDP, respectively) in the upper half of the DDS-i layer. Because both our L2 extensions are implemented in the lower half of the layer, we can take full advantage of CycloneDDS's integrated discovery, liveliness detection and security mechanisms for the small cost of needing to support the integrated addressing schemes (locators) and multicast communication. Since the underlying Layer 2 technology (Ethernet) supports both multicast and broadcast delivery, implementing this boils down to effectively managing the DPDK Poll-Mode Driver (PMD) or the XDP kernel data structures and efficiently converting CycloneDDS messages to Ethernet frames and back. The ability to use built-in discovery and lifecycle management thus allows us to support all related Quality of Service (QoS) settings out of the box and enables us to maximize standards compliance while minimizing additional user configuration and code maintenance.

Similarly, FastDDS also provides out of the box lifecycle management, including discovery, if we implement the transport interface. However, more attention needs to be paid to the implementation of the integrated addressing schemes: since FastDDS typically uses a combination of multicast messaging for discovery and unicast messaging for actual data transfer between peers, this complicates the handling of message routing compared to a simpler implementation, like CycloneDDS, which by default just relies on multicast. In a traditional UDP/TCP implementation, network sockets would have to be opened and closed to communicate with individual peers, and hence a significant portion of the transport interface is concerned with determining how to route messages

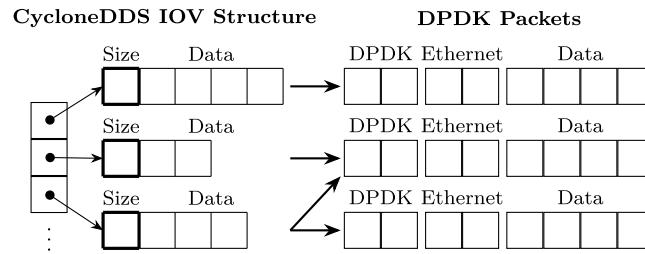


Fig. 4. The difference between the CycloneDDS/Linux IOV buffer format (left) and the DPDK packet buffer format (right). CycloneDDS buffers cannot be cleanly wrapped in DPDK packets, which is why the serialized data needs to be copied when traversing the extension on the outbound path.

across the physical and logical topology efficiently, i.e., which sockets to use and when. With the user-space networking extensions, we do not use sockets as we construct packets directly in the transport, and we also do not perform any special routing logic. However, it is still necessary to emulate/adapt some of the algorithms found in the UDP transport because the interface requires it.

We find that it is easy to make mistakes when implementing this routing logic, which originates from the fact that the API documentation is very light for this part of the transport interface. Anyone implementing new transports has to infer what each interface function does by interpreting existing implementations, e.g., by using the one for UDP as a reference. For certain components, it is not obvious what the function does or that it might serve multiple purposes — we were only able to resolve these issues after extensive debugging as FastDDS does not issue warnings in some cases where these functions are implemented slightly incorrectly. To remedy this problem for the future, we suggested improvements to the FastDDS documentation and source code, which are in the process of getting integrated into FastDDS itself. In this case, despite FastDDS offering a more formal C++ interface for implementing new transport technologies, it turns out that CycloneDDS extensions are easier to implement because CycloneDDS uses simpler logic for connection handling.

4.2. Serialization and buffer management

CycloneDDS's DDS-i layer contains the necessary logic to serialize and deserialize the samples that the user wants to transfer in the upper half of the DDS-i layer. This allows the L2 extensions to benefit from the built-in standard serialization mechanism, which future proofs the extension by allowing it to leverage vendor encoding interoperability, future CycloneDDS support for updated wire formats, and DDS extensible data types. It also relieves the extensions of taking care of data alignment, endianness, and other typical networking burdens. However, intercepting CycloneDDS communications below the message serialization sublayer also comes with a downside: it is not possible to control the buffers that CycloneDDS serializes its data to. This is problematic because CycloneDDS serializes its messages into several smaller buffers to pass them to the kernel via the IOV Linux kernel data structures (used by the syscalls `readv` and `writv`) to make passing the data to the kernel efficient (Fig. 4, left). The IOV data structure consists of an array of pointers to buffers that each have a size and a variable-length sequence of data octets. Usually, the outgoing packet will therefore not lie contiguously in memory.

Meanwhile, the packet buffer structure that DPDK expects is also rigid (Fig. 4, right), with each buffer containing DPDK managed metadata, followed by an Ethernet header managed by the DPDK-L2 extension and a fixed number of data octets for payload. To avoid needing to allocate the DPDK buffers when packets are sent and received, they are stored in a memory pool by our DPDK extensions. However, since both the CycloneDDS and the DPDK buffer formats are ultimately outside the extension's control, creating a copy of the data is unavoidable. When constructing DPDK frames to back the serialized CycloneDDS data stream, two possibilities can occur for each data fragment in the IOV structure:

1. If the data in the IOV fragment fits into the current DPDK frame, it simply needs to be copied from the IOV buffer into the DPDK frame's data buffer and then the frame is immediately transmitted through the DPDK driver.
2. If the data in the IOV buffer does not fit into the DPDK frame, the rest of the available space in the DPDK frame is filled with data from the IOV, the frame is queued for transmission, and a new frame is retrieved from the memory pool. This step is repeated until all data from the IOV fragment is packed into DPDK frames.

After one IOV fragment is successfully packed into the DPDK frame, we reiterate the algorithm above until all buffers in the IOV data structure have been processed. This replicates the fragmentation of the data stream into packets normally implemented in the kernel and NIC driver, which needs to be reversed for reconstruction of the original data stream on the receiving side by defragmentation and reordering. DPDK/XDP shift both these responsibilities from the kernel to the user.

FastDDS uses a very similar data structure to CycloneDDS's IOV data structure, called the network buffer vector. In contrast to CycloneDDS, though, FastDDS has mechanisms for querying the transport's maximum supported message size that will not lead to fragmentation. This makes it easier to control the buffer sizes of the incoming data on the publisher side, simplifying and thus accelerating the logic required to copy data buffers into DPDK packets since we are guaranteed to be able to fit FastDDS serialized messages into DPDK's `membufs`. On the subscriber side, we can avoid having to defragment packets, which reduces latency as well.

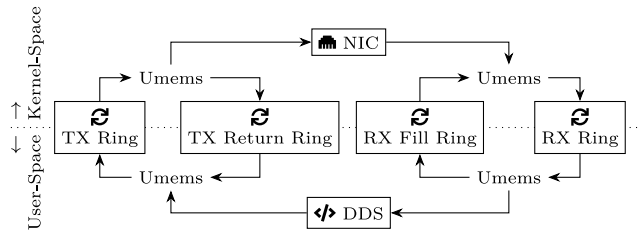


Fig. 5. The passing of data frames between user- and kernel-space in XDP. A umem is allocated by the user-space application and loaned to the kernel by either filling it with data and submitting it to the TX ring for sending, or by submitting it to the RX fill ring to let the kernel fill it with incoming packets. The kernel processes the requests and returns umems via the sibling ring.

However, although the logic is simpler, a single data copy is still necessary because FastDDS manages the buffers internally and cannot write directly into DPDK membufs, which are prerequisite to transferring data via DPDK.

A similar problem exists for XDP, where the extensions also need to manually assemble Ethernet frames in user-space. The frames are passed between the user-space application and the kernel or NIC driver via special blocks in user-space memory called umems. This works by using four single-producer single-consumer rings, two each for the TX path and the RX path (Fig. 5), which the umems traverse in cyclic fashion. For each path, one side is responsible for filling the buffers while the other side empties them. If hardware support is available, then the NIC can send packets directly to or from the umem buffer without needing to copy it first. However, both CycloneDDS and FastDDS cannot directly read from or write to the umem buffer as they do memory management internally, which is why at least one data copy is still necessary. If either DDS vendor offers an API to control serialization buffers in the future, zero-copy all the way from the serialization to the NIC hardware would become viable in both extensions, potentially enhancing performance further. A difference would be noticeable especially for larger samples, as the effort required to allocate and create copies of data increases with the frame size.

4.3. Independence from kernel data structures

In contrast to CycloneDDS's DDS-rt layer, which interacts very closely with the kernel, the DDS-i layer is still largely independent of the underlying kernel and physical environment and thus more portable. The relative distance in the DDS-i layer from the kernel makes circumventing the kernel for user-space networking easier, since we do not need to replicate much of the kernel's functionality, which we want to avoid for performance and code complexity reasons. It also allows us to bypass both the DDS-rt layer and the kernel simultaneously to interact directly with the NIC driver, further shortening the critical path for increased latency performance, while eliminating another potential source for unexpected latency.

However, the abstraction is not perfect, and for efficiency reasons there are exceptions where CycloneDDS hands kernel data structures all the way through to the DDS-i layer. One of these is the file descriptor (socket) used for network communication. CycloneDDS typically uses many sockets simultaneously for communication, and the DDS-i layer manages a collection of file descriptors to make this possible. Apart from configuring settings on the sockets, the CycloneDDS DDS-i layer repeatedly queries the DDS-rt layer for the next file descriptor that has data available for reading, which is implemented in the DDS-rt layer using the `select` system call. This allows CycloneDDS to enable one reading thread to manage data from multiple incoming sockets simultaneously, even across different transports. While the need to provide sockets is not an issue for the XDP-L2 extension, since the XSK API (XDP sockets) provides us with a pseudo socket that can be used with the aforementioned system calls, this does pose a problem to the DPDK-L2 extension, as there are no valid socket handles it could provide the DDS-i layer for managing.

There are two possible solutions to this problem: Either the DDS extension creates proxy file descriptors in the kernel or the need for having socket handles is removed altogether in favor of a different method of querying for available data. Since going through the kernel using proxy file descriptors defeats the purpose of using user-space networking technologies in the first place, we utilize the second option and modify the DDS-i layer to eliminate the need for file descriptors. Unfortunately, this is not possible without modifying the CycloneDDS core source code. However, it is possible to trick the DDS-i layer into automatically skipping most socket operations if it is told which transport to read from next ahead of time, before it attempts to determine it by itself. We implement this with a simple rule: when the DPDK/XDP extension is active, we assume that the extension is always the transport that should be read from. This simplification allows us to implement the DPDK-L2/XDP-L2 extension with only moderate modifications to the CycloneDDS core, and without compromising the regular functioning of CycloneDDS when the module is not loaded. However, this prevents regular transports from working side-by-side to the extensions. We deem this to be a relatively minor drawback though, as, e.g., DPDK requires the entire interface for itself anyway and thus a hybrid setup only works with at least a second network interface.

The same problem is not evident in FastDDS. Fortunately, FastDDS leaves the entire responsibility of managing sockets to the transport implementation, transferring data solely via read and write callbacks. This saves us the need to find workarounds for the lack of being able to provide sockets, which reduces the amount of modifications required to the FastDDS core implementation. In fact, the only necessary alterations in this case are the Quality of Service (QoS) parsers that determine which transport to use: these are extended for both CycloneDDS and FastDDS to include selection of the extensions and relevant option parsing.

Table 2
Transport implementations for CycloneDDS and FastDDS.

	CycloneDDS	FastDDS
Extensible Transport Implementation	✓	✓
Core Modifications	Minor (sockets)	Minimal
Easy Fragmentation Avoidance	~	✓
Extra Logic Required	Minimal	Some (locators)

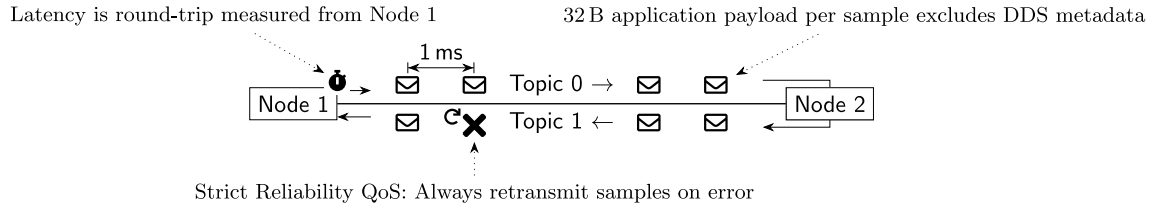


Fig. 6. DDS-Perf application and DDS Quality of Service (QoS) settings used for benchmarking DDS baseline implementations and the user-space networking extensions. Individual settings are modified as necessary to optimize for latency, sample rate and bandwidth.

Table 3
Important Application & QoS settings used in DDS-Perf [22].

Topics	Sample rate (1/s)	Size	Type	Randomization
2	1000 or unlimited	32 Bytes	Unkeyed	Partial
Reliability	History	Durability	Deadline	Latency Budget
Reliable	KEEP_ALL	VOLATILE	INFINITE	0

4.4. Implementation results

The aforementioned advantages (Table 2) of this design allow us to implement both of these extensions quite compactly. Because some functionalities in the extensions like packet handling are similar regardless of the actual user-space networking technology used, we can modularize the extensions, saving several hundred lines of code. This results in a total code base of under 3000 lines of extension code for the CycloneDDS extensions, with the XDP extension encompassing more code than the DPDK extension due to the need to manage the kernel BPF module. We deem it a particular success that the number of code changes inside the CycloneDDS core are minimal, with under 50 modified lines of code. The DPDK and XDP extensions can be compiled into CycloneDDS and activated at runtime using the same Quality of Service (QoS) settings file that also controls all other aspects of a DDS system's behavior, ensuring easy configurability using established DDS mechanisms. Because the FastDDS extensions are a port of the same functionality to FastDDS, they largely encompass the same logic and thus result in a similar code size to CycloneDDS, but even fewer core modifications are necessary. Furthermore, the optimizations we applied to both extensions afterwards did not significantly impact code size. Finally, with the completed extensions available in our deployed copies of CycloneDDS and FastDDS, we now quantify the performance benefits that can be achieved by benchmarking the implementation.

5. Benchmarks

To assess the intended advantages of user-space networking extensions for DDS against the traditional Linux networking stack, we require a benchmarking approach that is platform and DDS implementation agnostic. To achieve this, we leverage DDS-Perf [12] and adopt the same environment employed by the authors in a prior real-time study [22]. This maintains consistency in hardware and software configurations, ensuring a fair and comparable evaluation with the original findings.

5.1. Software setup

The DDS-Perf application is configured for two participants (Fig. 6): one generating samples via Topic 0 and the second relaying them back to their origin in Topic 1. A 32-Byte sample is generated every millisecond, which corresponds to a fixed sample rate of 1000 samples/s each way and 64 KiB/s total bandwidth usage due to bidirectionality. This remains comfortably below peak performance for all vendors, even on lower-end hardware [12], providing a safety margin akin to critical systems. All samples are unkeyed (a DDS feature we do not require for benchmarking), and partially randomized to make compression more difficult, emulating real-life conditions.

For comparability, we need to ensure that DDS' Quality of Service (QoS) settings are identical across all implementations (Table 3). To mirror the requirements of critical systems, we ask DDS to ensure strict reliability (the combination of RELIABLE reliability and KEEP_ALL history), as we want to ensure every sample is delivered. A viable alternative that is less demanding on the DDS implementation would be BEST_EFFORT reliability or a lower history setting for systems that can tolerate interim packet

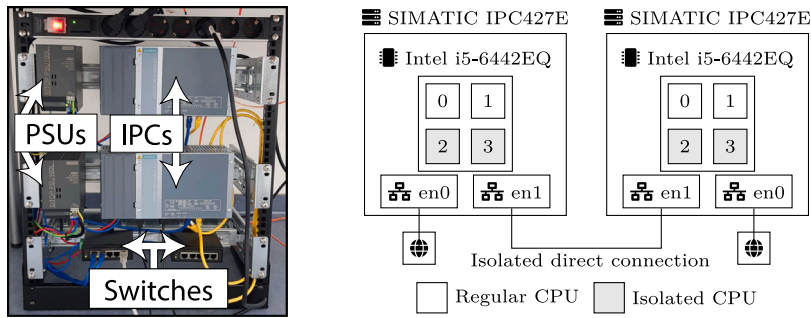


Fig. 7. The Siemens SIMATIC IPC427E systems for industrial IoT use (left side), each with the IPC box running a real-time tuned variant of Debian, a PSU, and the network. Schematic of the hardware setup (right side): Each system has two four-core CPUs with two isolated cores each connected by 2× Gigabit Ethernet switches.

losses. There are many more available QoS settings in DDS, with the OpenDDS manual [31] listing over 100 of them. Generally, we choose all unmentioned settings to be least constraining, allowing the DDS vendor to choose the optimal setting freely. Auxiliary data collection (such as the collection of system metrics) occurs in a separate process to avoid interfering with DDS-Perf or the underlying DDS library. Orchestration occurs outside the test system by a dedicated manager node.

5.2. Hardware setup

We evaluate performance on two dedicated Siemens IPC units (Fig. 7) running Debian 11.5. These systems use a PREEMPT_RT kernel (Linux jupiter 5.18.0-0.deb11.4-rt-amd64) for low latency scheduling. Each unit has an Intel i5-6442EQ @ 1.90GHz CPU, with two general purpose cores and two isolated cores (via the `isolcpus` kernel option) useful for interference-free computation. This prevents any tasks from being scheduled on them unless the corresponding task affinity is set first, but load balancing does work between the cores if the affinity allows it. The systems do not use Non-Maskable Interrupts (NMIs). Each system has 8 GB of DDR4 RAM, with swap turned off to prevent latency caused by page faults, and is interconnected by one general purpose and one isolated dedicated network for use by DDS. The dedicated network is wired directly point-to-point, with no intermittent switches to avoid additional packet delay variation. All components are rack-mounted, using passive cooling. With the system environment in place, we must now reproduce the original study's DDS configuration.

5.3. Setup reproduction

In the original investigation on real-time tuning of DDS [22], the authors iteratively refined their system configuration to improve performance of the vendors with DDS-Perf. To make our results comparable, we replicate the final configuration from Section 6.E, requiring us to perform the same hardware and software tuning. Both the core isolation and the network isolation were reproduced on the hardware side and the relevant QoS and kernel adjustments were made on the software side. We verified the equivalency of our setup by comparing with the original performance data. Each profile shown contains at least 1 million samples, and while we show only one trial, each experiment was validated at least three times.

In contrast to our previous study on user-space networking for DDS [21], we now adjust sample rate and bandwidth measurements to account for packet loss. While our strict reliability QoS settings (Reliable Reliability QoS + Keep All History QoS + unlimited maximum blocking time on Data Writer write queue overflow [32]) theoretically require DDS to deliver messages completely reliable, in practice samples still get dropped. We suspect this originates mainly from the liveliness detection mechanism, which can immediately disconnect clients if the sample processing rates or response times are not sufficient to send the necessary heartbeats between participants. This problem becomes more significant with the addition of FastDDS-DPDK and FastDDS-XDP, therefore we considered it now necessary to compensate for the difference by adjusting for the number of dropped packets. For this reason, the absolute measurement results are not directly comparable to the original study, but the overall performance trends remain the same.

5.4. Results: Latency

Latencies and latency bounds are the primary concern in critical real-time systems. These are optimal when the workload's characteristic overall system utilization is low, which means sending small samples and limiting publishing frequency to a rate of 1000 samples/s. From the results in Figs. 8 and 9, we observe a significant round-trip time improvement for both user-space networking technologies. For CycloneDDS-DPDK, we achieve a mean latency of 74 us, equivalent to a reduction of 31% on our test system when compared to the regular implementation of CycloneDDS. FastDDS-DPDK improves on the mean latency performance of regular FastDDS as well, with 38% performance improvement. As expected due to the larger involvement of the kernel, the performance of XDP is slightly worse than DPDK for both CycloneDDS and FastDDS, at 87 us and 86 us respectively, but this still

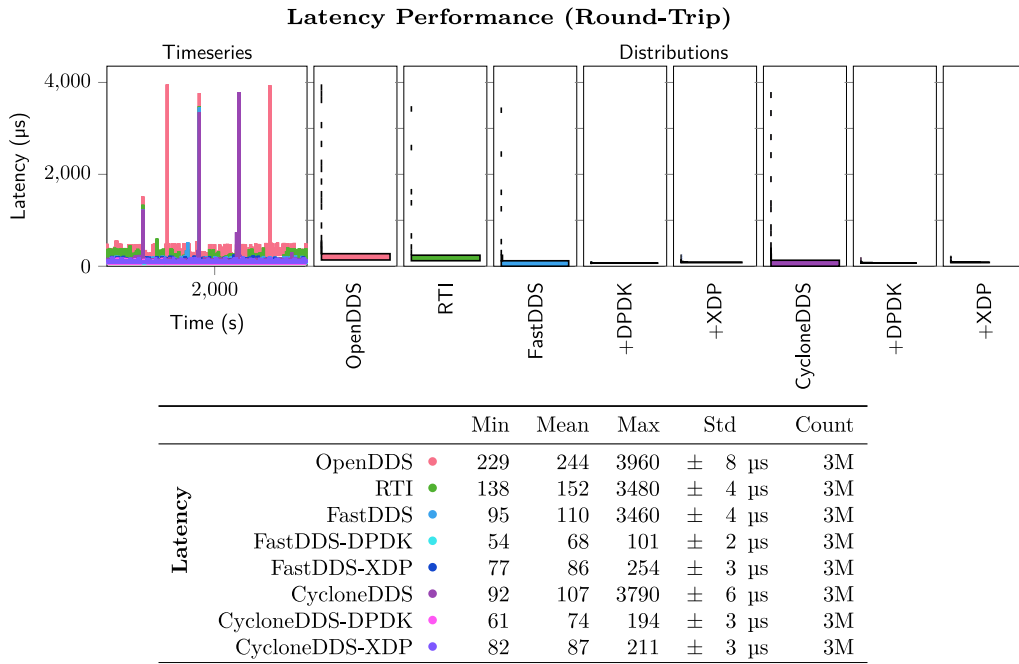


Fig. 8. Latency analysis of the original four vendors and each two user-space networking extensions, with a time-series (left) and vertical histograms (right). The user-space networking extensions outperform the other systems in all latency metrics, with the DPDK extensions slightly surpassing the XDP extensions. Furthermore, even though the baseline performance of FastDDS and CycloneDDS is practically equivalent, the FastDDS extensions generally offer slightly better latency results. Apart from the latency reduction, the performance variability is also significantly reduced.

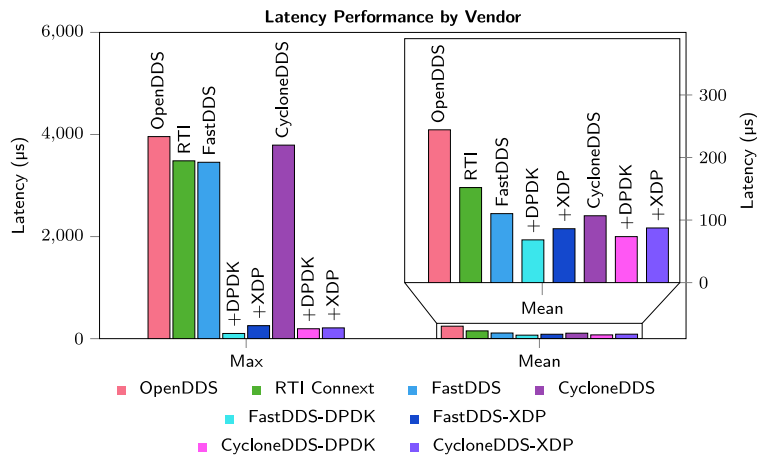


Fig. 9. Latency performance summary for maximum observed latency/latency bound (left) and average latency performance (right). The CycloneDDS extensions offer 95% improved latency bounds and at least 18%–31% improved average latency over the traditional DDS systems, while the FastDDS extensions match or even slightly surpass this with up to 97% reduced latency bounds.

corresponds to a latency reduction of 18% and 22% over the status quo. A typical round trip for the worst performing system, OpenDDS, will now take around 3x the time it takes to complete a CycloneDDS-DPDK round trip even after OpenDDS was tuned for latency. The better mean latency performance of our extensions allows us to effectively deliver sub-0.1-millisecond round-trips, or sub-0.05-millisecond derived one-way latency, exceeding what was possible using conventional DDS systems.

Previously, we also highlighted the importance of minimizing latency outliers. Our results indicate progress here as well, with a maximum observed latency of 194 µs and 211 µs for the CycloneDDS extensions, respectively. Where previously we observed rare but high magnitude outliers of up to 35x the mean latency for CycloneDDS, with CycloneDDS-DPDK this margin is reduced to only 2.6x. While this still exceeds the latency bound of DPDK's native tool `l2reflect` [22], it is nevertheless a sizable 95%

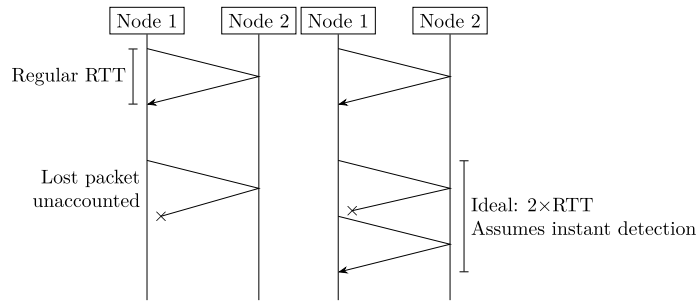


Fig. 10. Theoretically optimal latency bounds in presence of a single transmission failure under best effort and reliable quality of service. While best effort can ignore late and dropped packets, reliable transmission is always at the disadvantage of having to apply error correction, which introduces latency.

improvement on the status quo. The corresponding worst latency in 1 million packets observed (with 0.0001% chance of occurring) is 184 us for CycloneDDS-DPDK and 203 us for CycloneDDS-XDP, demonstrating considerable performance-reliability improvements. This is also seen in a substantially reduced packet delay variation (PDV), which is more than halved to 50% and 57% lower PDV than regular CycloneDDS, respectively. For the first time on this hardware/software combination, we have thus achieved a round-trip time latency bound well below 1 millisecond for around 3 million packets.

The picture is slightly different for FastDDS. While FastDDS-DPDK shows competitive performance in both mean and maximum observed latency, with FastDDS-DPDK exhibiting the best latency bound overall at just 101 us, the port of the XDP plugin from CycloneDDS to FastDDS produced more mixed results: the mean performance of FastDDS-XDP is on par with the original XDP implementation for CycloneDDS at 86 us, but the maximum observed latency is slightly worse than the other user-space networking extensions. Originally, we had issues with the way that the ported XDP plugin interacts with FastDDS that caused it to fall back on the kernel more often, thus leading to maximum observed latencies more akin to the native FastDDS implementation. However, further optimization in regard to latency behavior enabled us to achieve a better latency bound closer to the other technologies, nevertheless DPDK still shows an edge over XDP in terms of latency performance for both CycloneDDS and FastDDS.

Despite these improvements, we still see a substantial difference between *l2reflect*'s latency bound of $1.25\times$ the mean latency (max $125\mu\text{s}$) and CycloneDDS-DPDK's $2.6\times$ /CycloneDDS-XDP's $2.4\times$. This is relatively easily explained through differing measurement conditions: whereas our DDS implementations are configured for strict reliability, *l2reflect* never retransmits packets in case of failure. Any packets that are dropped are simply not accounted for in *l2reflect*, making the optimal performance unrealistic if we cannot afford to lose data. Thus, the comparison turns out to be rather unfair: whenever a retransmission is necessary, one entire additional roundtrip is incurred to deliver the data a second time. Therefore, the best round-trip time that can be achieved in the presence of packet loss via retransmission is twice the regular round-trip time (Fig. 10), and this is only possible if the lost packet can be detected instantly.

It can be argued that even a lower bound of $2\times\text{RTT}$ is optimistic for maximum latency, since it requires the packet to be retransmitted instantly when the original packet did not arrive within $1\times$ the average RTT. Since there is a variation in packet delay (quantified by the Packet Delay Variation, PDV), such a retransmission strategy would cause many unnecessary retransmissions if packets are just slightly delayed rather than lost entirely. Thus, additional time is often allocated to allow late packets to arrive, further raising the optimal lower bound. Thus, a maximum round trip time of $2.6\times$ the mean latency for CycloneDDS-DPDK is suddenly a lot closer to the optimum than it may appear if one considers the overhead incurred by error correction.

Since the best latency bound (FastDDS-DPDK) is below $2\times$ the round-trip time, with 101 us corresponding to just $1.5\times$ the mean latency, we can infer that there are no retransmissions in the trials for FastDDS-DPDK. So while FastDDS-DPDK might look to perform the best on paper, the presence of unpredictable packet loss raises the same question of comparability: in theory any implementation can arbitrarily drop samples that arrive beyond a set tolerance or target latency bound. This is enabled by the fact that DDS implementations still drop packets despite strict reliability. Since these cannot be accounted for, strategic or coincidental sample loss can theoretically make the latency bound look artificially better than it should be under reliable transmission circumstances. If instead it were possible to force truly reliable delivery, the question would remain on how many errors/retransmissions to include into each measurement trial to make the comparison fair. We thus find it difficult to conclusively evaluate which implementation offers superior latency bound performance when they are this close to optimal due to limitations in the experimental setup. However, we can conclude that all user-space networking extensions improve significantly on the baseline's latency bound and packet delay variability.

Nevertheless, it would still theoretically be possible to reduce the chance of latency outliers and their magnitudes further even if reliable message delivery is required. Redundancy can help mitigate the need for a second round-trip in case of a single error, albeit at the cost of additional resources. We can reasonably infer that our measurements for CycloneDDS-DPDK and CycloneDDS-XDP contain only isolated transmission errors, as the need to retransmit twice would already push the maximum latency to at least $3\times\text{RTT}$ but the observed maximum latency is significantly lower in both cases. Since we are only dealing with single packet errors, double redundant transmission should be sufficient to mitigate isolated errors and allow reliable DDS communications to be competitive even with the best effort latency results of *l2reflect*. However, this is more of a theoretical possibility: in practice,

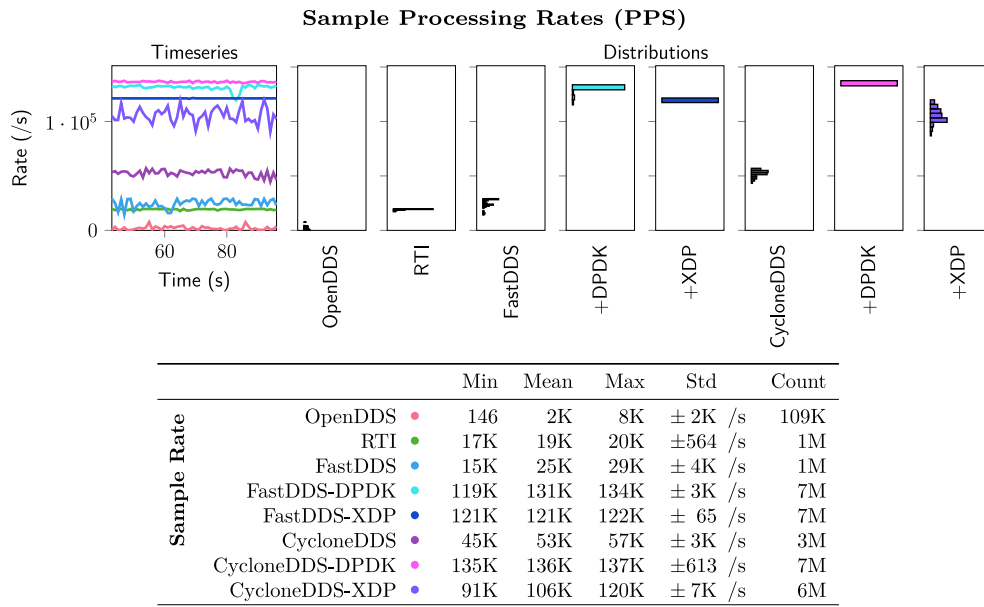


Fig. 11. Maximum sample rates achieved by the DDS implementations at 32 Bytes/sample. All user-space networking extensions are able to reach extremely high sample rates, in most cases significantly outperforming 100K samples/s. DPDK leads over the XDP extensions, with CycloneDDS-XDP showing the most variability as it approaches its peak processing capacity. All four variants outperform the regular DDS implementations, with OpenDDS achieving the lowest peak sample rate overall and only CycloneDDS approaching a comparable level of performance.

DDS does not directly offer built-in support for redundant transmission. Even if it did, eventually two errors will occur in close proximity to each other despite the very low error rates inherent in point-to-point Ethernet, thus exceeding the error correction introduced by redundancy and making retransmission necessary again, which in turn introduces additional latency. It is here that we encounter the fundamental conflict that is an issue in many real-time communications systems: unconditional reliability in the face of transmission errors and bounded latency are two fundamentally conflicting goals.

In summary, we conclude that the latency behavior we observe is close to optimal: the average round-trip time actually slightly exceeds *l2reflect*'s previously observed performance [22] and the maximum observed latency can only be further reduced by sacrificing reliability or introducing redundancy. However, real-time systems do not only require latency performance, we additionally need to consider throughput to assess overall efficiency.

5.5. Results: Sample rate throughput

While latency is our primary motivation in real-time behavior analysis, other performance benefits of the CycloneDDS extensions are also of interest. For throughput, there are two factors we can consider: bandwidth throughput and sample rate throughput. Each of these requires certain workload characteristics to exhaust the system capacity, so we introduce two new configurations: one optimal for sample rate and one for bandwidth. If we remove the target throughput of 1000 samples/s, we can determine the peak sample rate for our testbed (Fig. 11), achieved through small samples at high frequency. Previously, CycloneDDS-DPDK reached the highest mean sample rate of the original user-space networking extensions in the original study. With the new optimizations applied, CycloneDDS-DPDK again offers the top sample rate throughput at 136K samples/s, constituting a 160% increase over regular CycloneDDS with 53K samples/s, while simultaneously reducing performance variability. Whereas CycloneDDS-XDP did not reach the same level of performance as CycloneDDS-DPDK or even regular CycloneDDS in the original study, even though both extensions relied on much of the same code, the subsequent optimizations and the compensation for packet loss equalize this difference: CycloneDDS-XDP now competes in sample-rate performance to the other user-space networking extensions. We still infer that the process of reading packets one-by-one hurts XDP performance, as again we see slightly lower performance on XDP while reading individual packets does not have such a big performance penalty for DPDK.

Both FastDDS plugin implementations also significantly outperform the original FastDDS baseline, now achieving beyond 100K sample/s processing rates. We attribute this to some optimizations we were able to perform while porting the plugins to FastDDS to tune throughput: for one, the design of the FastDDS API allows us to more efficiently process several samples simultaneously on the receiving path, since we can receive multiple Ethernet frames at once and feed them individually into the FastDDS callback. This design allows us to take advantage of the fact that user-space networking extensions are designed to handle packets in batches and thus leverage memory locality, and especially XDP seems to profit from the fact that we can now handle multiple packets at once. The other optimization we were able to apply is a counter-intuitive one: we artificially slow down packet processing rates when they become excessive on the publisher side. This became necessary because the publishing rate on the sender node for XDP started

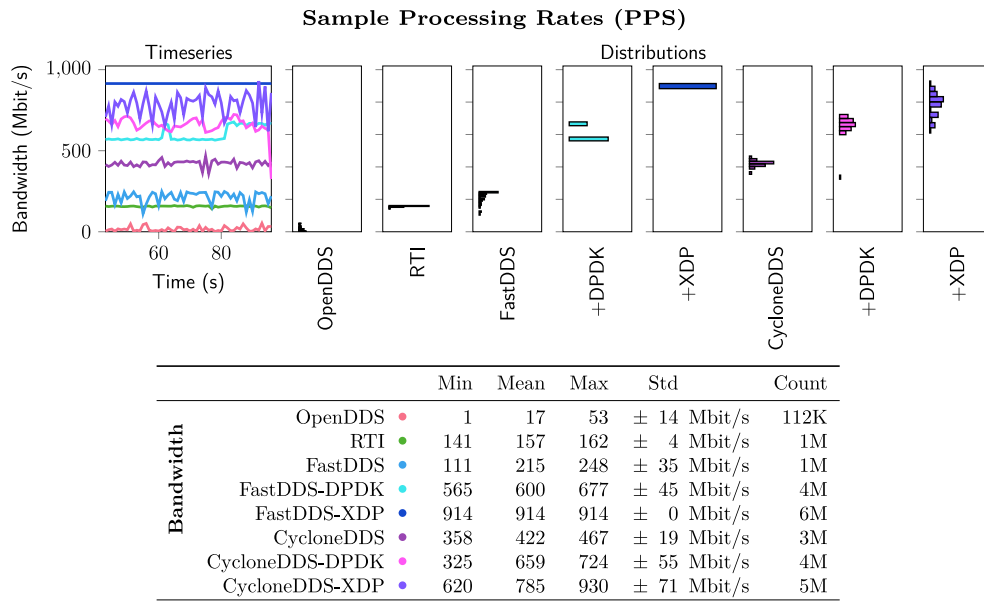


Fig. 12. Maximum achievable DDS bandwidth for the extensions at 1KB/sample. Contrary to peak sample rate performance, DPDK does not outperform XDP in terms of bandwidth throughput, where FastDDS-XDP shows the most promise. Again, optimizations allowed for a significant boost of performance, but the increased performance generally implies higher performance variability.

to overwhelm the subscriber, thus leading to frequent disconnects and very few samples transferred successfully. This optimization causes fewer packets to be sent, but simultaneously significantly improves on the packet drop rate, thus a lower load and target throughput ends up leading to a higher goodput (i.e. application-layer throughput). It also allows the FastDDS extensions to provide for much more stable packet processing rates evident in the significantly reduced range and standard variation. The FastDDS-DPDK plugin does not require this mechanism, but we activate it where beneficial for performance.

Both optimizations could theoretically also be applied back to the CycloneDDS extensions. However, the first optimization (batch-processing on the receiving path) is slightly more difficult to implement in the CycloneDDS API, as it would require a caching mechanism. We were able to backport the second optimization (adaptive rate publishing) back to the CycloneDDS extensions, improving performance to the level we now observe in Fig. 11. However, finding the right level of artificial slow-down requires parameter tuning, which ends up being hardware and vendor/plugin-specific, and needs to be repeated for each combination of software, plugins and hardware. Thus, optimization potential will inherently vary. This can be observed for CycloneDDS-XDP, which appears close to its peak processing capacity due to the higher performance variability observed. We are of the opinion though that further future optimizations could, however, still significantly boost processing rates for the other implementations as variability is still lower. Overall, we managed to raise the bar regarding packet processing rates to at least 201% of the best non-extension baseline, now surpassing 100K samples/s with every user-space networking extension.

5.6. Results: Bandwidth throughput

A similar effect is also visible for the maximum achievable bandwidth (Fig. 12), achieved by transferring large samples at high frequency. CycloneDDS already performs quite well by default, with 421.5 Mbit/s bandwidth saturating a reasonable portion of the Gigabit Ethernet interface. At 1KB payload/sample, the per-sample overheads inside of DDS paired with the fact that packet loss rates are significant at full load still bottleneck the throughput, which is why we cannot saturate the physically available bandwidth. With the optimized FastDDS-DPDK accelerated extension, we were able to improve throughput by 56%, averaging at 658.7 Mbit/s. CycloneDDS-XDP is able to surpass this slightly with 784.6 Mbit/s, but at the cost of significantly more variance in the throughput. In terms of bandwidth, CycloneDDS-DPDK and FastDDS-DPDK are on par with each other, with the original CycloneDDS-DPDK at the advantage as FastDDS-DPDK only reaches a mean performance of 600.2 Mbit/s. Through the timing optimizations in FastDDS-XDP, we have not only increased the available bandwidth throughput to 914.3 Mbit/s, the highest performance we observe among all implementations, but we also significantly improved on the bandwidth variance. A variance this low is made possible by a relatively conservative adjustment of the integrated rate limiter, sacrificing a slight amount of performance in favor of stability. Relative to the FastDDS baseline, FastDDS-XDP's bandwidth performance improves upon the status quo by a factor of 2.2×.

5.7. Results: Summary

In summary, CycloneDDS and FastDDS user-space networking extensions can significantly enhance various key performance metrics (Fig. 13 for CycloneDDS extensions and Fig. 14 for FastDDS) compared to their original DDS implementations based on the

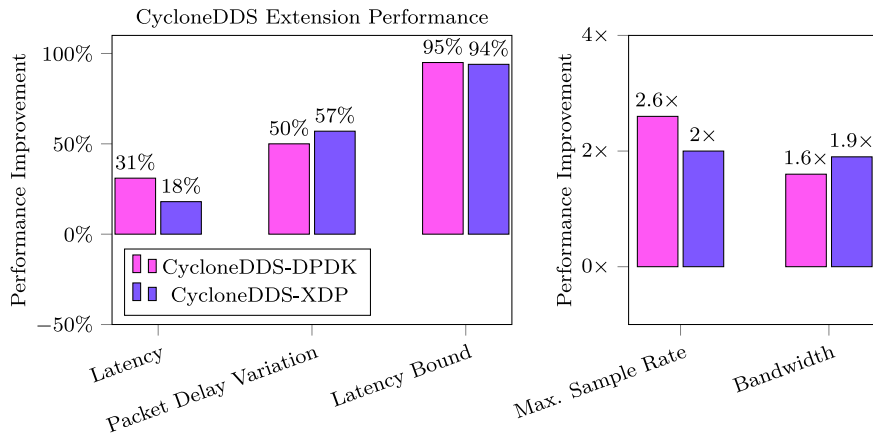


Fig. 13. The relative performance gains for the CycloneDDS user-space networking extensions (higher is better). While we deem the decreased latency bound as the most important benefit for real-time systems, other key performance metrics can also be significantly improved through careful tuning of the extensions.

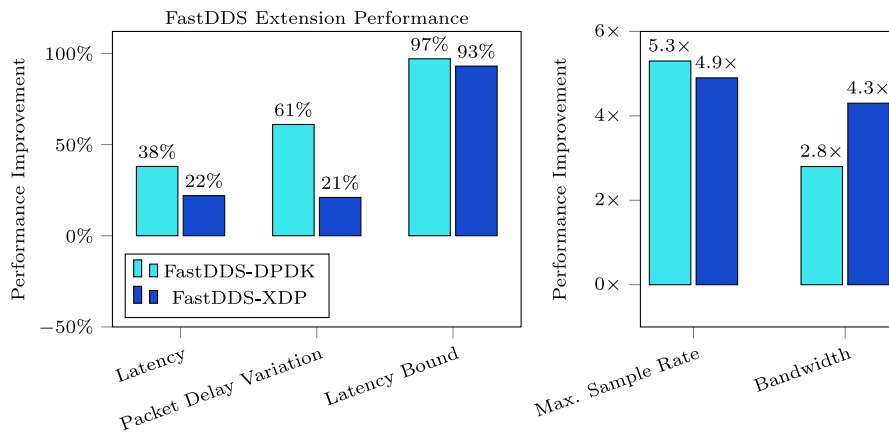


Fig. 14. The relative performance gains for the FastDDS user-space networking extensions (higher is better). FastDDS-DPDK outperforms regular FastDDS in every performance metric significantly. FastDDS-XDP also has improves on the baseline, showing the highest overall bandwidth performance improvement, but the benefits in terms of average latency and packet delay variation are less pronounced.

traditional Linux networking stack. The largest improvement is evident in the drastically reduced latency bound, which is one of the most important performance metrics for industrial applications. Since CycloneDDS already offers very competitive performance out of the box and after tuning, this also means that the original CycloneDDS-based DPDK/XDP extensions already outperformed competing DDS implementations before we applied further optimizations. With the new FastDDS extensions, we were able to achieve comparable performance metrics, and we find that DPDK slightly surpasses XDP in latency performance. In terms of maximum sample rate and bandwidth throughput, through the introduction of new optimizations we were able to outperform the baseline multiple times over, demonstrating the potential of fine-grained optimization for the out-of-kernel communication paths. User-space networking technologies still have their usual downsides due to their invasive nature, but the performance benefits still makes their adoption attractive for specialized deployments. XDP is promising as it integrates DPDK-like latency performance advantages seamlessly without infrastructure disruption. For critical performance needs, we recommend the DPDK extension, which further enhances latency performance and peak sample rate throughput, compensating for deployability drawbacks. Whereas we previously recommended XDP for its less invasive nature but suggested DPDK for raw performance needs (based off of the data we had for the CycloneDDS extensions at the time), the picture is now less clear, as we observe that the XDP extensions often match and occasionally outperform the DPDK extensions, especially in terms of bandwidth performance. We thus conclude that the underlying technology for user-space networking itself may not be the most significant limiting factor. Instead, the true potential of these technologies is only unlocked through careful tuning — and this applies not only to the DDS extensions, but equally to the core implementations themselves.

6. Discussion & future work

While embedded hard real-time systems still have the edge when it comes to real-time communications, the gap between hard and soft real-time systems is closing. Our CycloneDDS and FastDDS-based user-space networking extensions contribute to DDS as a performance-oriented communication middleware catching up, but there are other developments in the DDS world working in the same direction.

IEEE 802.1 Time Sensitive Networking (TSN) is a family of standards that brings real-time capabilities and determinism to Ethernet networks. With compatible hardware (network interfaces and switching equipment), it is possible to schedule traffic and allocate network streams that can be used interference free, which will further reduce unpredictable latency by providing network delay bounds at the hardware level. While TSN capabilities are currently implemented within the Linux kernel, there is also the possibility of combining TSN capabilities with user-space networking technologies to get both deterministic packet handling and the performance benefits from circumventing the kernel. At the time of writing, discussions are ongoing about XDP's suitability for TSN delivery, but no proof of concept is yet available [33].

Another possibility for increasing efficiency is removing the need for data copying on the transmission and reception paths, which is especially cumbersome and thus latency inducing for large message sizes. This is theoretically possible because DDS controls the data structures that the application uses to communicate, they are defined at compile time in Interface Definition Language (IDL) structs. Thus, data could be laid out with the necessary headroom for packet headers in DPDK mbufs or XDP's umem buffers so that the packets may be filled by the application, which just sees its regular IDL-defined data structure. The headers would then be filled in-place and the packet could be directly written to wire by the NIC. While this promises performance advantages for simple data structures, a zero-copy approach with no serialization quickly turns complicated when less trivial structures utilizing features like variable length messages are involved. Our research suggests that the need for dynamic memory allocation because of variable length messages hurts the ability to offer reliable latency bounds for some vendors. Additionally, laying out the data for complete zero-copy support would require extensive modifications on all levels of CycloneDDS/FastDDS code, an expensive endeavor that will likely outweigh the benefits of fewer data movement. Efforts to this end could perhaps leverage existing CycloneDDS/FastDDS support for shared memory communication, as support for shmem necessarily already requires handling of non-exclusively owned data buffers.

Depending on the deployment, security considerations can also be an important concern. With DDS user-space networking, there are two aspects to consider: Maintaining the security of the data being transmitted and ensuring that the additional privileges of the user-space networking technologies cannot compromise the operation of adjacent systems. Since DDS has built in support for encryption that is also supported by our CycloneDDS and FastDDS extensions, data security is supported out of the box through the DDS-security module. This makes the extensions just as secure as the base implementation itself in this regard, as the extension only operates on serialized/encrypted data when security support is active. As an additional measure, industrial systems will often employ dedicated networks where physical access can be controlled. The matter of privileges is more difficult to secure though, as the user-space networking technology essentially has promiscuous access to the network card and is therefore able to read all traffic, even traffic that is unrelated to the application. Ensuring that the packet processing in the extensions cannot be hijacked, e.g., through a bug in the user application, would therefore be desirable in the future. For XDP, this would e.g. be possible by verifying the integrity of the packet filter when loading it into the kernel. These security lock-down mechanisms are subject to future research.

Finally, we observe that some performance penalties occur due to the handling of single packets when communicating with the NIC, a problem likely present in all extensions but more visible in some than others. Reading packets one-by-one offers the best latency performance, which is the main metric we are interested in. However, the more congestion starts occurring when the throughput is increased, the more the overhead of one-by-one processing starts becoming visible. CycloneDDS's DDS-i layer is only capable of fetching a single packet from the extensions at a time, but theoretically the extension could fetch multiple packets from the NIC and buffer extraneous packets for later processing in CycloneDDS, thus reducing the communication with the NIC. Such a buffering mechanism would likely introduce some additional latency at low load due to larger transfer requests from the NIC as well as additional memory management, but especially for XDP a performance benefit for sample rate and bandwidth could likely be realized. In FastDDS, due to the more suitable design of the API on the subscriber-side, we were able to batch process packages on the subscriber, which boosted performance. However, neither DDS vendor API makes it easy to do the same on the publisher, thus additional logic would be required to manually gather packets into batches on the publisher. This would likely boost throughput further at the cost of introducing latency.

7. Conclusion

In this paper, we introduced user-space networking technologies for use with the DDS message-oriented middleware standard. Analyzing previous studies, it was found that DPDK's user-space networking technology outperforms four DDS implementations using the traditional Linux networking stack in latency bounds, showing a potential for DDS performance benefits by leveraging a user-space networking stack. We originally implemented two extensions for the high performance DDS implementation CycloneDDS based on the popular user-space networking technologies DPDK and XDP, and later added two more extensions based on FastDDS. These extensions handle all Layer 2 Ethernet packet management within the respective DDS library and communicate directly with the NIC, circumventing the kernel on the communication hot-path. Using the CycloneDDS-DPDK and CycloneDDS-XDP extensions as accelerators, we demonstrated that both a significant reduction in mean latency is achievable (DPDK: 31%, XDP: 18%), that the

packet delay variation can be more than halved and the maximum latency bounds can be reduced drastically for both extensions to under 250 μ s ($\sim$ 95% improvement). Finally, in the case of DPDK the peak sample rate and bandwidth can be further increased by 160% and 56%, respectively. For the FastDDS extensions, we see a similar latency pattern, although the packet delay variation is not as strongly improved. In terms of sample rate and bandwidth however, FastDDS-DPDK and FastDDS-XDP outperform the FastDDS baseline by 4.9 $\times$, 5.3 $\times$ and 4.3 $\times$, 2.8 $\times$, respectively. Using these user-space networking extensions, it is now possible to provide soft latency bounds of well under one millisecond on the DDS platform, which was not previously achievable on our industrial edge system. With the continuing effort to bring DDS to more timing-critical applications, we think that DDS with user-space networking technologies strikes a good balance between the offered performance and the necessary specialization on the spectrum of out-of-the-box messaging middleware over Linux IP on one end to the use of dedicated microcontroller communication on the other. We hope that the community benefits from our contributions to CycloneDDS and FastDDS and we invite users to try out the DPDK and XDP extensions on their systems.

CRedit authorship contribution statement

Vincent Bode: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Carsten Trinitis:** Funding acquisition. **Martin Schulz:** Writing – review & editing, Supervision. **David Buettner:** Resources. **Tobias Preclik:** Resources.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Vincent Bode reports a relationship with Siemens AG that includes: funding grants. An educational license for the commercial DDS implementation RTI Connex was provided by the vendor free of charge for evaluation. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] P. Naur, B. Randell, Software Engineering: Report of a Conference Sponsored by the Nato Science Committee, Garmisch, Germany, 7th–11th October 1968, Newcastle University, 1969.
- [2] Object Management Group, OMG data distribution service (DDS), 2015, [Online]. Available: <https://www.omg.org/spec/DDS/1.4/PDF>, Standard Specification.
- [3] Y. Maruyama, S. Kato, T. Azumi, Exploring the performance of ROS2, in: Proceedings of the 13th International Conference on Embedded Software, ISBN: 9781450344852, 2016, <http://dx.doi.org/10.1145/2968478.2968502>.
- [4] RTI Inc., Operational Avionics Layer (OPAL) - IAI Aviation Group, 2020, [Online]. Available: https://www.rti.com/hubfs/_Collateral/Customer_Snapshots/rti-customer-snapshot-iai-aviation.pdf.
- [5] Object Computing, Inc., Achieving Global Health with Leading-Edge Diagnostics, 2021, [Online]. Available: <https://objectcomputing.com/case-studies/a-healthier-future>.
- [6] I. Calvo, F. Pérez, I. Etxeberria-Agiriano, O.G. de Albéniz, Designing high performance factory automation applications on top of DDS, Int. J. Adv. Robot. Syst. 10 (4) (2013) 205, <http://dx.doi.org/10.5772/56341>.
- [7] I. Calvo, F. Pérez, O.G. de Albeniz, I. Etxeberria-Agiriano, Towards a OMG DDS communication backbone for factory automation applications, in: ETFA2011, 2011, pp. 1–4.
- [8] C.S.V. Gutiérrez, L.U.S. Juan, I.Z. Ugarte, V.M. Vilches, Real-time Linux communications: an evaluation of the Linux communication stack for real-time robotic applications, 2018, arXiv preprint [arXiv:1808.10821](https://arxiv.org/abs/1808.10821).
- [9] A. Belkhir, M. Pepin, M. Bly, M. Dagenais, Performance analysis of DPDK-based applications through tracing, J. Parallel Distrib. Comput. (ISSN: 0743-7315) 173 (2023) 1–19, <http://dx.doi.org/10.1016/j.jpdc.2022.10.012>, [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0743731522002271>.
- [10] T. Høiland-Jørgensen, J.D. Brouer, D. Borkmann, J. Fastabend, T. Herbert, D. Ahern, D. Miller, The express data path: Fast programmable packet processing in the operating system kernel, in: Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies, CoNEXT'18, Association for Computing Machinery, New York, NY, USA, ISBN: 9781450360807, 2018, pp. 54–66, <http://dx.doi.org/10.1145/3281411.3281443>.
- [11] Object Management Group, OMG data distribution service (DDS), 2004, [Online]. Available: <https://www.omg.org/spec/DDS/1.0/PDF>, Standard Specification.
- [12] V. Bode, D. Buettner, T. Preclik, C. Trinitis, M. Schulz, Systematic analysis of DDS implementations, in: Proceedings of the 24th International Middleware Conference, in: Middleware'23, Association for Computing Machinery, Bologna, Italy, ISBN: 9798400701771, 2023, pp. 234–246, <http://dx.doi.org/10.1145/3590140.3629118>.
- [13] S. Profanter, A. Tekat, K. Dorofeev, M. Rickert, A. Knoll, OPC UA versus ROS, DDS, and MQTT: Performance evaluation of industry 4.0 protocols, in: 2019 IEEE International Conference on Industrial Technology, ICIT, 2019, pp. 955–962.
- [14] Z. Kang, A. Dubey, Evaluating DDS, MQTT, and ZeroMQ under different IoT traffic conditions, 2020.
- [15] X. Chen, X. Kong, Y. Ling, X. Cao, DDS performance evaluation for PREEMPT_RT Linux, in: 2021 International Conference on Computer, Blockchain and Financial Development, CBFDD, 2021, pp. 84–89.
- [16] S. Solpan, K. Kucuk, DDS-XRCE standard performance evaluation of different communication scenarios in IoT technologies, EAI Endorsed Trans. Internet of Things (2022) <http://dx.doi.org/10.4108/eetiot.v8i4.2691>.

- [17] R. Serrano-Torres, M. García-Valls, P. Basanta-Val, Virtualizing DDS middleware: Performance challenges and measurements, in: 2013 11th IEEE International Conference on Industrial Informatics, INDIN, 2013.
- [18] P. Bellavista, A. Corradi, L. Foschini, A. Pernaflini, Data distribution service (DDS): A performance comparison of OpenSplice and RTI implementations, in: 2013 IEEE Symposium on Computers and Communications, ISCC, 2013, pp. 377–383.
- [19] Object Management Group, DDS extensions for time sensitive networking (DDS-TSN), 2023, [Online]. Available: <https://www.omg.org/spec/DDS-TSN/1.0/Beta1/PDF>.
- [20] E. Guijarro Cameros, L. Chan, How DDS and TSN are driving interoperability and performance in automotive systems, *ATZelectronics Worldw.* 17 (10) (2022) 52–55.
- [21] V. Bode, C. Trinitis, M. Schulz, D. Buettner, T. Preclik, Adopting user-space networking for DDS message-oriented middleware, in: 2024 IEEE International Conference on Pervasive Computing and Communications (PerCom), 2024, pp. 36–46.
- [22] V. Bode, C. Trinitis, M. Schulz, D. Buettner, T. Preclik, DDS implementations as real-time middleware – A systematic evaluation, in: 2023 IEEE 29th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA), 2023, pp. 186–195.
- [23] E. Borde, G. Haïk, V. Watine, L. Pautet, Really hard time developing hard real-time, in: 2nd National Workshop on Control Architecture of Robots, 2007.
- [24] A.M. Cheng, *Real-Time Systems: Scheduling, Analysis, and Verification*, John Wiley & Sons, 2003.
- [25] I. Mugarza, J. Parra, E. Jacob, Analysis of existing dynamic software updating techniques for safe and secure industrial control systems, *Int. J. Saf. Secur. Eng.* 8 (1) (2018) 121–131.
- [26] Petr Skocik, System calls overhead, 2017, [Online]. Available: <https://stackoverflow.com/questions/23599074/system-calls-overhead>.
- [27] DPDK, DPDK users's guide, 2023, [Online]. Available: https://doc.dpdk.org/guides/linux_gsg/.
- [28] T. Zhang, L. Linguaglossa, M. Gallo, P. Giaccone, D. Rossi, FloWatcher-DPDK: Lightweight line-rate flow-level monitoring in software, *IEEE Trans. Netw. Serv. Manag.* 16 (3) (2019) 1143–1156, <http://dx.doi.org/10.1109/TNSM.2019.2913710>.
- [29] M. Karlsson, B. Töpel, The path to DPDK speeds for AF XDP, in: *Linux Plumbers Conference*, 2018.
- [30] T. Begin, B. Baynat, G. Artero Gallardo, V. Jardin, An accurate and efficient modeling framework for the performance evaluation of DPDK-based virtual switches, *IEEE Trans. Netw. Serv. Manag.* 15 (4) (2018) 1407–1421.
- [31] Object Computing, Inc., OpenDDS developer's guide, 2021, [Online]. Available: <https://download.objectcomputing.com/OpenDDS/previous-releases/OpenDDS-3.17.pdf>.
- [32] D. Seltz, Top 10 Reasons for Dropped DDS Messages, Real Time Innovations, 2020, [Online]. Available: <https://www.rti.com/blog/top-10-reasons-for-dropped-dds-messages>.
- [33] XDP-Project, Investigate if XDP can be used for TSN, 2019, [Online]. Available: <https://github.com/xdp-project/xdp-project/issues/2>.